



**UNIVERSIDAD PRIVADA DE TACNA**

**FACULTAD DE INGENIERIA**

**Escuela Profesional de Ingeniería de Sistemas**

**INFORME DE LABORATORIO**  
**“Alineación de Fútbol”**

Curso: Programación II

Docente: Ing. Breno Luque Zuñiga

**Tarqui Ramos, Sebastián Emir Andree (2022075482)**

**Tacna – Perú**  
**2026**



## INDICE

|                                                                      |    |
|----------------------------------------------------------------------|----|
| <b>I. INFORMACIÓN GENERAL</b> .....                                  | 3  |
| - <b>Objetivos:</b> .....                                            | 3  |
| - <b>Equipos, materiales, programas y recursos utilizados:</b> ..... | 3  |
| <b>II. MARCO TEORICO</b> .....                                       | 3  |
| <b>III. PROCEDIMIENTO</b> .....                                      | 4  |
| <b>IV. ANALISIS E INTERPRETACION DE RESULTADOS</b> .....             | 9  |
| <b>V. CUESTIONARIO</b> .....                                         | 10 |
| <b>CONCLUSIONES</b> .....                                            | 10 |



## **INFORME DE LABORATORIO**

### **Alineación de Fútbol**

#### **I. INFORMACIÓN GENERAL**

- **Objetivos:**
  - Implementar una aplicación de escritorio en C# Windows Forms usando Programación Orientada a Objetos.
  - Crear una clase Jugador con encapsulamiento correcto (propiedades con get/set).
  - Gestionar una colección dinámica de objetos mediante List<Jugador> en memoria.
  - Aplicar las operaciones CRUD (Agregar, Modificar, Eliminar) sobre la lista de jugadores.
  - Mostrar la información registrada en un DataGridView con columnas personalizadas.
- **Equipos, materiales, programas y recursos utilizados:**
  - Computadora con Windows 10/11
  - Visual Studio 2022 Community Edition
  - Lenguaje de programación: C# (.NET Framework / .NET 6+)
  - Windows Forms (WinForms) como framework de interfaz gráfica
  - Captura del diseño del formulario proporcionada como referencia

#### **II. MARCO TEORICO**

##### **Programación Orientada a Objetos (POO)**

La POO es un paradigma de programación que organiza el código en torno a objetos, que son instancias de clases. Sus pilares fundamentales son: abstracción, encapsulamiento, herencia y polimorfismo.

##### **Clase y Objeto**

Una clase es una plantilla que define atributos (datos) y métodos (comportamientos). Un objeto es una instancia concreta de esa clase. En este proyecto, Jugador es la clase y cada jugador registrado es un objeto.

##### **Encapsulamiento**

El encapsulamiento protege los datos de un objeto, exponiendo solo lo necesario a través de propiedades públicas. En C# esto se logra con propiedades { get; set; }, lo que permite validar o controlar el acceso a los campos internos.

Colecciones — List<T>



List<T> es una colección genérica dinámica de .NET que permite agregar, eliminar y buscar elementos sin necesidad de definir un tamaño fijo. Métodos clave usados: Add(), RemoveAt(), Exists() e indexación directa por posición.

### DataGridView

Es un control de Windows Forms que muestra datos en formato de tabla. Con AutoGenerateColumns = false y DataPropertyName se enlaza directamente a las propiedades de los objetos de la lista, refrescando la vista cada vez que se reasigna DataSource.

## III. PROCEDIMIENTO

Se creó un proyecto Windows Forms en Visual Studio 2022 y se siguieron los pasos a continuación:

1. Se creó la clase Jugador.cs con cuatro propiedades: Nombre, NumeroCamiseta, Posicion y Condicion, junto con un constructor que recibe los cuatro parámetros.

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6
7  namespace Trabajo_en_clase_19_05_2026_Tarqui_Sebastian
8  {
9      4 referencias
10     public class Jugador
11     {
12         1 referencia
13         public string Nombre { get; set; }
14         6 referencias
15         public int NumeroCamiseta { get; set; }
16         1 referencia
17         public string Posicion { get; set; }
18         1 referencia
19         public string Condicion { get; set; }
20
21         1 referencia
22         public Jugador(string nombre, int numeroCamiseta, string posicion, string condicion)
23         {
24             Nombre = nombre;
25             NumeroCamiseta = numeroCamiseta;
26             Posicion = posicion;
27             Condicion = condicion;
28         }
29     }
30 }
```

2. Se diseñó el formulario Form1 en el Designer con los controles indicados en la siguiente tabla:

| Control  | Nombre      | Descripción                                                                  |
|----------|-------------|------------------------------------------------------------------------------|
| TextBox  | txtNombre   | Campo para ingresar el nombre del jugador                                    |
| TextBox  | txtNumero   | Campo para ingresar el número de camiseta                                    |
| ComboBox | cmbPosicion | Lista desplegable con posiciones: Portero, Defensa, Mediocampista, Delantero |



| Control      | Nombre       | Descripción                                        |
|--------------|--------------|----------------------------------------------------|
| ComboBox     | cmbCondicion | Lista desplegable con condición: Titular, Suplente |
| Button       | btnAgregar   | Agrega un nuevo jugador a la lista                 |
| Button       | btnEliminar  | Elimina el jugador seleccionado                    |
| Button       | btnModificar | Modifica los datos del jugador seleccionado        |
| Button       | btnLimpiar   | Limpia todos los campos del formulario             |
| DataGridView | dgvJugadores | Muestra la lista de jugadores registrados          |
| Label        | lblContador  | Muestra el total de jugadores registrados          |

Form1

### Alineación de Futbol

Nombre

Numero

Posicion

Condicion

**Agregar** **Eliminar** **Modificar** **Limpiar**

|   | N° Camiseta | Nombre | Posición | Condición |
|---|-------------|--------|----------|-----------|
| * |             |        |          |           |

Contador:

3. En Form1.cs se declaró List<Jugador> listaJugadores y la variable indiceSeleccionado para rastrear la fila activa del grid.

```
11 namespace Trabajo_en_clase_19_05_2026_Tarqui_Sebastian
12 {
13     3 referencias
14     public partial class Form1 : Form
15     {
16         private List<Jugador> listaJugadores = new List<Jugador>();
17         private int indiceSeleccionado = -1;
```

4. En el constructor se llamó a CargarComboBoxes() para poblar cmbPosicion y cmbCondicion, y a ConfigurarDataGridView() para definir las columnas manualmente.

```
25     1 referencia
26     private void CargarComboBoxes()
27     {
28         cmbPosicion.Items.AddRange(new string[] { "Portero", "Defensa", "Mediocampista", "Delantero" });
29         cmbCondicion.Items.AddRange(new string[] { "Titular", "Suplente" });
30     }
31
32     1 referencia
33     private void ConfigurarDataGridView()
34     {
35         dgvJugadores.AutoGenerateColumns = false;
36         dgvJugadores.Columns.Clear();
37
38         dgvJugadores.Columns.Add(new DataGridViewTextBoxColumn { HeaderText = "Nº Camiseta", DataPropertyName = "NumeroCamiseta", Width = 90 });
39         dgvJugadores.Columns.Add(new DataGridViewTextBoxColumn { HeaderText = "Nombre", DataPropertyName = "Nombre", Width = 160 });
40         dgvJugadores.Columns.Add(new DataGridViewTextBoxColumn { HeaderText = "Posición", DataPropertyName = "Posicion", Width = 110 });
41         dgvJugadores.Columns.Add(new DataGridViewTextBoxColumn { HeaderText = "Condición", DataPropertyName = "Condicion", Width = 100 });
42     }
43
44     3 referencias
45     private void ActualizarGrilla()
46     {
47         dgvJugadores.DataSource = null;
48         dgvJugadores.DataSource = listaJugadores;
49         lblContador.Text = "Contador: " + listaJugadores.Count;
```

5. Se implementó ValidarCampos() para verificar que ningún campo esté vacío y que el número sea un entero entre 1 y 99.

```
49     private bool ValidarCampos()
50     {
51         if (string.IsNullOrEmpty(txtNombre.Text))
52             { MessageBox.Show("Ingrese el nombre del jugador."); return false; }
53
54         if (!int.TryParse(txtNumero.Text, out int num) || num < 1 || num > 99)
55             { MessageBox.Show("Ingrese un número de camiseta válido (1-99)."); return false; }
56
57         if (cmbPosicion.SelectedIndex < 0)
58             { MessageBox.Show("Seleccione una posición."); return false; }
59
60         if (cmbCondicion.SelectedIndex < 0)
61             { MessageBox.Show("Seleccione una condición."); return false; }
62
63         return true;
64     }
```

6. En btnAgregar\_Click se validó, se verificó que el número de camiseta no esté duplicado con Exists(), se creó el objeto Jugador y se añadió a la lista.

```
77 private void btnAgregar_Click(object sender, EventArgs e)
78 {
79     if (!ValidarCampos()) return;
80
81     int numero = int.Parse(txtNumero.Text);
82
83     // Verificar número de camiseta duplicado
84     if (listaJugadores.Exists(j => j.NumeroCamiseta == numero))
85     {
86         MessageBox.Show("Ya existe un jugador con ese número de camiseta.");
87         return;
88     }
89
90     var jugador = new Jugador(
91         txtNombre.Text.Trim(),
92         numero,
93         cmbPosicion.SelectedItem.ToString(),
94         cmbCondicion.SelectedItem.ToString()
95     );
96
97     listaJugadores.Add(jugador);
98     ActualizarGrilla();
99     LimpiarFormulario();
100 }
```

7. En btnModificar\_Click se actualizaron directamente las propiedades del objeto en la posición indiceSeleccionado de la lista.

```
102 private void btnModificar_Click(object sender, EventArgs e)
103 {
104     if (indiceSeleccionado < 0)
105     { MessageBox.Show("Seleccione un jugador para modificar."); return; }
106
107     if (!ValidarCampos()) return;
108
109     int numero = int.Parse(txtNumero.Text);
110
111     // Verificar duplicado excluyendo el mismo jugador
112     if (listaJugadores.Exists(j => j.NumeroCamiseta == numero &&
113         listaJugadores.IndexOf(j) != indiceSeleccionado))
114     {
115         MessageBox.Show("Ya existe otro jugador con ese número de camiseta.");
116         return;
117     }
118
119     listaJugadores[indiceSeleccionado].Nombre = txtNombre.Text.Trim();
120     listaJugadores[indiceSeleccionado].NumeroCamiseta = numero;
121     listaJugadores[indiceSeleccionado].Posicion = cmbPosicion.SelectedItem.ToString();
122     listaJugadores[indiceSeleccionado].Condicion = cmbCondicion.SelectedItem.ToString();
123
124     ActualizarGrilla();
125     LimpiarFormulario();
126 }
```

8. En btnEliminar\_Click se solicitó confirmación con MessageBox y se ejecutó RemoveAt().

```
128 private void btnEliminar_Click(object sender, EventArgs e)
129 {
130     if (indiceSeleccionado < 0)
131     { MessageBox.Show("Seleccione un jugador para eliminar."); return; }
132
133     var confirm = MessageBox.Show("¿Eliminar al jugador seleccionado?", "Confirmar",
134     MessageBoxButtons.YesNo, MessageBoxIcon.Question);
135
136     if (confirm == DialogResult.Yes)
137     {
138         listaJugadores.RemoveAt(indiceSeleccionado);
139         ActualizarGrilla();
140         LimpiarFormulario();
141     }
142 }
```

9. En dgvJugadores\_CellClick se captura la fila seleccionada y se cargan sus datos en los controles del formulario.

```
155 private void dgvJugadores_CellClick(object sender, DataGridViewCellEventArgs e)
156 {
157     if (e.RowIndex < 0 || e.RowIndex >= listaJugadores.Count) return;
158
159     indiceSeleccionado = e.RowIndex;
160     var j = listaJugadores[indiceSeleccionado];
161
162     txtNombre.Text = j.Nombre;
163     txtNumero.Text = j.NumeroCamiseta.ToString();
164     cmbPosicion.SelectedItem = j.Posicion;
165     cmbCondicion.SelectedItem = j.Condicion;
166 }
```

10. Se probaron todas las operaciones: agregar jugadores con distintas posiciones, editar un jugador existente, eliminar y limpiar el formulario.



**Alineación de Fútbol**

Nombre

Numero

Posicion

Condicion

**Agregar** **Eliminar** **Modificar** **Limpiar**

| N° Camiseta | Nombre    | Posición | Condición |
|-------------|-----------|----------|-----------|
| 78          | Sebastian | Defensa  | Suplente  |

Contador: Contador: 1



#### IV. ANALISIS E INTERPRETACION DE RESULTADOS

##### **¿Qué indican los resultados?**

La aplicación funcionó correctamente en todas las operaciones probadas. El DataGridView refleja en tiempo real cualquier cambio en la lista gracias al patrón de reasignar DataSource = null y luego DataSource = listaJugadores. El contador muestra siempre el número exacto de jugadores registrados.

Las validaciones impidieron ingresar datos vacíos o números de camiseta duplicados, lo que garantiza integridad de los datos sin necesidad de una base de datos.

##### **¿Qué se ha encontrado?**

Se confirmó que List<T> es suficiente para manejar colecciones pequeñas en memoria con todas las operaciones CRUD. El encapsulamiento en la clase Jugador facilitó el acceso limpio a los datos desde el formulario. Separar la lógica (Form1.cs) del diseño (Designer) mejoró la legibilidad del código. También se encontró que la validación de duplicados con Exists() y una lambda es más clara y compacta que un bucle manual.



## V. CUESTIONARIO

### 1. ¿Qué es una clase en POO?

Es una plantilla que define atributos y métodos para crear objetos. En el proyecto, Jugador es la clase principal.

### 2. ¿Para qué sirve el encapsulamiento?

Para proteger los datos de un objeto. Se usó con propiedades públicas (get/set) en la clase Jugador.

### 3. ¿Qué ventaja tiene usar List<T> sobre un arreglo?

List<T> es dinámica: crece y decrece en tiempo de ejecución. Permite Add(), Remove() y Exists() sin definir tamaño fijo.

### 4. ¿Por qué no se usó base de datos?

La consigna lo prohibía. Toda la información vive en memoria con List<Jugador>, lo que simplifica el desarrollo y es suficiente para el alcance del ejercicio.

### 5. ¿Cómo se evita duplicar números de camiseta?

Con listaJugadores.Exists(j => j.NumeroCamiseta == numero) antes de agregar o modificar.

## CONCLUSIONES

- La clase Jugador con propiedades encapsuladas fue suficiente para modelar la entidad del dominio sin complejidad innecesaria.
- List<Jugador> en memoria reemplazó perfectamente a una base de datos para el alcance del ejercicio, con operaciones CRUD simples y directas.
- Mantener Form1.cs solo con lógica y dejar el diseño al Designer hace el código más ordenado y fácil de mantener.
- La validación de campos y la detección de duplicados son esenciales para la integridad de los datos, incluso sin base de datos.
- Windows Forms sigue siendo una herramienta práctica y rápida para construir aplicaciones de escritorio académicas con POO en C#.